

TimeLens AI

Adaptive Task-Duration Planner: Technical Report on the Minimum Viable Prototype

Vansh Jain (1024030770) **Devyang** (1024030765) **Bhavik** (1024030772)

Department of Computer Engineering
Thapar Institute of Engineering and Technology

September 2026

Abstract

TimeLens AI is a local, single-user proof of concept that corrects a person's task-duration estimates using their own completion history, lays out a deterministic daily timeline from those corrected durations, and re-sequences remaining work when a task runs late. Rather than a trained machine-learning model, the prototype uses a transparent statistical baseline: a blended category bias, a time-of-day multiplier, and an interruption overhead term, so that every prediction can be explained to the user in plain language. This report documents the implemented system: its architecture, prediction formula, scheduling rules, data model, demonstration workflow, evaluation figures observed from the seeded demo, and known limitations, and outlines the scope for a production-grade extension.

1 Overview

Task planners typically treat a user's self-reported duration estimate as ground truth and build a schedule directly from it. In practice, people are systematically biased estimators: some categories of work reliably run long, performance varies with time of day, and interruptions add overhead that is rarely budgeted for. TimeLens AI targets this gap at prototype scale: it is a single Next.js application, with no authentication, cloud service, or external API, that demonstrates the complete loop of *predict* → *schedule* → *adapt* → *learn* using a local SQLite database.

2 Problem Statement

Conventional task managers function as passive storage systems for user-supplied durations. This produces four recurring failure modes that the prototype is designed to illustrate:

- **Unmitigated estimation bias.** Users under- or over-estimate durations in category-specific, measurable ways, and planners never correct for it.
- **Assumed uniform productivity.** Output varies with time of day, yet planners treat every hour as equally productive.
- **Uncaptured interruption cost.** Context switches and interruptions add real overhead that is absent from the original estimate.
- **Static schedule degradation.** Once a task overruns, the remainder of the day is left unchanged, compounding the delay.

TimeLens AI addresses these with an explainable statistical correction rather than a black-box model, prioritizing transparency and demonstrability over predictive sophistication at this stage.

3 System Architecture

The prototype runs as a single Next.js (App Router) application; there is no separate backend, external API, authentication layer, or cloud dependency.

- `src/app`: server-rendered dashboard and server actions.
- `src/components`: interactive dashboard components.
- `src/lib`: Prisma client, prediction logic, scheduling logic, analytics, and seed fixtures.
- `src/types`: framework-independent planning types.
- `prisma`: SQLite schema and seed entry point.
- `tests`: focused Vitest calculation tests.

The browser invokes Next.js server actions to perform mutations; those actions call Prisma directly and revalidate the dashboard. There is no client-side API layer to secure or version, which keeps the prototype's surface area small while still exercising the full predict-schedule-adapt loop end to end.

3.1 Technology Stack

Layer	Technology
Application framework	Next.js (App Router), TypeScript
Data access	Prisma ORM
Persistence	SQLite (local file)
Testing	Vitest

4 Prediction Model

Each new task's duration is corrected using completed `TaskHistory` records from the same category:

$$\text{prediction} = \text{estimate} \times \text{category bias} \times \text{time-of-day factor} + \text{interruption overhead} \quad (1)$$

- **Category bias** is the median ratio of actual to estimated duration for that category, blended toward a default of 1.10 until at least six matching records exist.
- **Time-of-day factor** is 0.95 for morning, 1.15 for afternoon, and 1.05 for evening tasks.
- **Interruption overhead** adds three minutes per average historical interruption for the category, capped at fifteen minutes.
- The result is rounded and constrained to a practical range. Confidence is reported as LOW, MEDIUM, or HIGH, rising as matching history accumulates.

4.1 Worked Example

The seeded *Technical Report* history carries a category bias of 1.25 and an average of three interruptions. A 60-minute task scheduled for a 2:00 PM preferred start is therefore corrected as:

$$60 \times 1.25 \times 1.15 + 9 = 95.25 \approx 95 \text{ minutes} \quad (2)$$

Figure 1 shows the two multiplicative factors that drive this correction across the categories and times of day used in the seeded demo data.

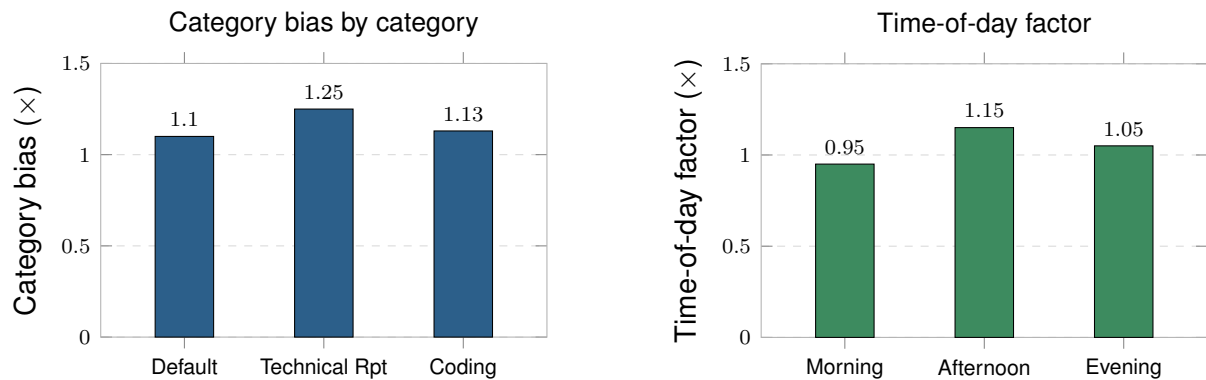


Figure 1: Multiplicative correction factors used in the prediction formula, drawn from the seeded demo data (Technical Report and Coding categories) and the fixed morning/afternoon/evening multipliers.

5 Scheduling Engine

Once a duration is predicted, the task is placed into a deterministic daily timeline:

- Completed tasks are frozen; their saved schedule fields are never modified.
- Pending tasks are ordered by deadline, with priority breaking ties within a two-hour window.
- The plan begins at 9:00 AM and runs tasks sequentially using predicted (not estimated) minutes.
- A task is flagged *at risk* if its scheduled end falls after its deadline.

5.1 Delay Simulation

The prototype can simulate a fixed 35-minute delay on any task. This shifts that task's end time and propagates the same 35-minute shift to every later pending task, demonstrating adaptive re-sequencing rather than a static, one-shot plan.

6 Data Model

The local SQLite database contains exactly two application models.

Model	Purpose
Task	Current title, category, estimate, prediction, priority, deadline, schedule, status, actual duration, interruption count, and timestamps.
TaskHistory	Immutable completion snapshots used for future predictions and analytics.

Priority is one of LOW, MEDIUM, or HIGH; task status is PENDING or COMPLETED.

7 MVP Feature Set

- Create, delete, and complete tasks without authentication or a separate backend.
- Store task data and completion history in local SQLite through Prisma.
- Explain each predicted duration with category, time-of-day, and interruption factor cards.
- Build a deterministic plan from 9:00 AM using deadline, priority, and predicted duration.
- Simulate a 35-minute delay and shift the delayed task plus every later pending task.

- Record actual duration on completion and update the lightweight analytics panel.
- Restore consistent demonstration data on demand with *Reset demo*.

8 Demonstration Workflow

The recorded demonstration follows a fixed six-step sequence:

1. Open the seeded dashboard and point out the four planned tasks and the learning panel.
2. Enter *Technical Report*, a 60-minute estimate, a 3:00 PM deadline, and an afternoon preferred start (the form defaults to 2:00 PM).
3. Add the task and show its 95-minute predicted duration together with its factor cards and explanation.
4. Inspect its placement in the daily timeline, built from corrected rather than raw durations.
5. Apply a +35-minute delay to the new report and confirm every later pending task shifts by 35 minutes.
6. Enter an actual duration, mark the task complete, and observe the completed-task count and analytics panel update. Use *Reset demo* to repeat from the original state.

9 Evaluation and Observed Metrics

Because the prototype uses an explainable baseline rather than a trained model, evaluation focuses on *workload correction* and *historical accuracy* rather than classical ML validation metrics. The figures below are those surfaced live on the seeded demo dashboard.

Dashboard metric	Seeded demo value
Pending tasks	4
Original estimate (sum)	235 min
Predicted workload (sum)	284 min
Extra time detected	+49 min
Mean absolute error (completed tasks)	13 min
Completed tasks recorded	12

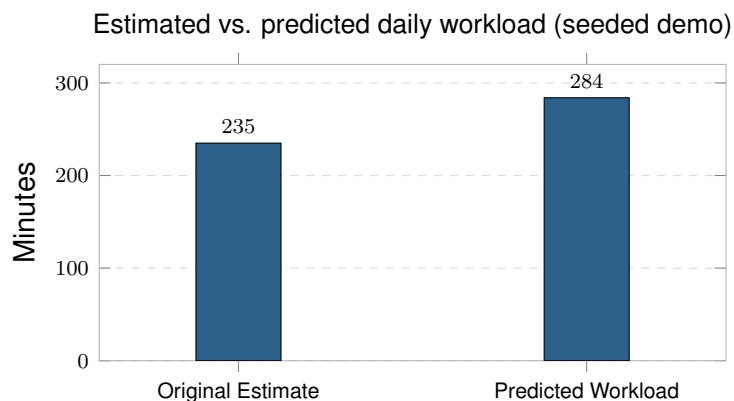


Figure 2: TimeLens AI's correction adds 49 minutes of previously unbudgeted time across the four pending demo tasks, surfacing the gap between what users estimate and what the historical model predicts.

The mean absolute error of 13 minutes across 12 completed tasks is reported directly by the analytics panel and reflects the gap between each task's predicted and actual duration under

the current category-bias and time-of-day model; it is a descriptive statistic over the seeded history rather than a held-out test-set metric, since the MVP does not yet separate training and evaluation data.

10 Known Limitations

- The planner builds a single sequential day and does not model breaks, calendars, working-hour limits, dependencies, or recurring tasks.
- The prediction is deliberately a transparent statistical baseline; there is no model training or personalization across multiple users.
- Preferred start is used only for the initial time-of-day prediction; it is not a persisted scheduling constraint, since the MVP schema is intentionally minimal.

11 Phase 2 / Production Roadmap

- Multiple planning days, availability windows, and task dependencies.
- More detailed interruption capture and trend visualizations.
- A user-controlled calibration view and richer model evaluation.
- Optional export and calendar integrations, while keeping the core planner understandable.
- Longer term: accounts, cloud persistence, gradient-boosted or other trained ML models, and notification delivery.

12 Setup and Reproduction

1. Requires Node.js 20 or later.
2. `npm install`
3. `npm run setup`: generates the Prisma client, creates/updates the SQLite database, and loads fresh demo data.
4. `npm run dev`: then open <http://localhost:3000>.
5. `npm test`: runs the Vitest calculation checks.

To recreate the recorded demonstration while the local server is running: `npm run demo:record`, then `npm run demo:mp4`, then `npm run db:seed` to restore the original demo state.

13 Summary

TimeLens AI's MVP demonstrates, in a single local codebase, that a fully explainable statistical correction (category bias, time-of-day adjustment, and interruption overhead) is enough to show a complete predict-schedule-adapt-learn loop without authentication, cloud infrastructure, or a trained model. The seeded demo's own figures (a 13-minute mean absolute error over 12 completed tasks and a 49-minute workload correction across four pending tasks) provide concrete, reproducible evidence for the core idea, while the documented limitations set clear boundaries for what remains prototype-only versus what a production system would need to add.